
Neura Modbus Interface

Neura Robotics GmbH

Jul 11, 2025

CONTENTS:

1	Remote Program Control	1
1.1	Modbus Configuration	1
1.2	Adding an External YAML File	1
1.3	Configuration Example	1
1.4	Modbus Configuration	2
2	Modbus Input Registers (Read-Only)	3
3	Modbus Holding Registers (Read/Write)	5
3.1	General Purpose Registers (Custom Use)	7

REMOTE PROGRAM CONTROL

This chapter describes how to remotely control robot programs using the **Remote Program Launcher** and **Modbus Configuration**.

Robots provide a remote program launcher that allows you to control the execution of robot programs. These include the programs listed in the GUI, as well as external programs.

1.1 Modbus Configuration

The **Modbus** page in the Interface section of the Robot GUI provides a page to configure and activate/deactivate the Modbus server.

On the Modbus configuration, you can choose between configurations for the Remote Program Launcher. In this release, the configuration is provided with configuration files formatted in YAML. The format is simple and lists the numbers to be associated with GUI or external programs.

1.2 Adding an External YAML File

To add an external YAML file from a USB drive (**SecureSync**): 1. Insert the **SecureSync** USB into the robot system. 2. Create a folder named **modbus** inside the **SecureSync** USB drive. 3. Copy the YAML file into this folder. 4. Ensure that the YAML file follows the same format as shown below. 5. In the **Robot GUI**, navigate to the **Upload from USB** option and select it to upload the external programs.

1.3 Configuration Example

Below is an example of a YAML configuration for Modbus and program mapping:

```
modbus_config:
  discrete_inputs: 100
  coils: 100
  holding_registers: 3000
  input_registers: 3000
  server_type: TCP # or RTU
  port: /dev/ttyUSB0 # only for RTU
  baudrate: 9600 # only for RTU

program_map:
```

(continues on next page)

(continued from previous page)

```
1: "Program_001"  
2: "Program_002"  
3: "Program_003"  
4: "Program_004"  
5: "Program_005"
```

The *modbus_config* section specifies the configuration for Modbus communication, including the server type (TCP or RTU), port, and baud rate for RTU communication.

The *program_map* section maps program numbers to their respective names for easier identification and control.

1.4 Modbus Configuration

The **Modbus** page in the Interface section of the Robot GUI provides a page to activate/deactivate the Modbus server.

Modbus TCP Communication - The Modbus TCP server runs on **port 5222**. - Ensure that all Modbus clients are configured to connect using this port.

MODBUS INPUT REGISTERS (READ-ONLY)

Ad- dress	Description	Type	Notes
1000	Robot Mode	UINT16	0 = UNKNOWN, 1 = INIT, 2 = STANDBY, 3 = TEACH, 4 = AUTO, 255 = ERROR
1001	Power On/Off Status	UINT16	0 = OFF, 1 = ON
1002	Program Running Status	UINT16	0 = Not Running, 1 = Running
1003	Program Execution Status	UINT16	0 = Succesful, 1 = Failed
1006	Fault Status	UINT16	0 = No Fault, 1 = Fault
1007	Collision Detected Status	UINT16	0 = No Collision, 1 = Collision
1008	Digital Input DI_0	USINT Bit	IO state of DI_0 (digital input 0)
1009	Digital Input DI_1	USINT Bit	IO state of DI_1 (digital input 1)
1010	Digital Input DI_2	USINT Bit	IO state of DI_2 (digital input 2)
1011	Digital Input DI_3	USINT Bit	IO state of DI_3 (digital input 3)
1012	Digital Input DI_4	USINT Bit	IO state of DI_4 (digital input 4)
1013	Digital Input DI_5	USINT Bit	IO state of DI_5 (digital input 5)
1014	Digital Input DI_6	USINT Bit	IO state of DI_6 (digital input 6)
1015	Digital Input DI_7	USINT Bit	IO state of DI_7 (digital input 7)
1016	Analog Input Type	USINT Register	Indicates the type of analog input: 0 = Volt- age, 10 = Ampere
1017- 1024	Analog Inputs AI_0-AI_7	REAL Element	Continuously updated with the state of AI_0 to AI_7
1025- 1032	Analog Outputs AO_0-AO_7	REAL Element	Continuously updated with the state of AO_0 to AO_7
1042- 1043	Joint angle A1	REAL, Float32 (LE)	Continuously updated with the current joint angle of A1 (little-endian)
1044- 1045	Joint angle A2	REAL, Float32 (LE)	Continuously updated with the current joint angle of A2 (little-endian)
1046- 1047	Joint angle A3	REAL, Float32 (LE)	Continuously updated with the current joint angle of A3 (little-endian)
1048- 1049	Joint angle A4	REAL, Float32 (LE)	Continuously updated with the current joint angle of A4 (little-endian)
1050- 1051	Joint angle A5	REAL, Float32 (LE)	Continuously updated with the current joint angle of A5 (little-endian)
1052- 1053	Joint angle A6	REAL, Float32 (LE)	Continuously updated with the current joint angle of A6 (little-endian)
1054- 1055	Joint angle A7	REAL, Float32 (LE)	Continuously updated with the current joint angle of A7 (little-endian)

continues on next page

Table 1 – continued from previous page

Ad- dress	Description	Type	Notes
1056- 1057	TCP pose X	REAL, Float32 (LE)	Continuously updated with the current TCP X position (little-endian)
1058- 1059	TCP pose Y	REAL, Float32 (LE)	Continuously updated with the current TCP Y position (little-endian)
1060- 1061	TCP pose Z	REAL, Float32 (LE)	Continuously updated with the current TCP Z position (little-endian)
1062- 1063	TCP pose A (Rotation about X-axis)	REAL, Float32 (LE)	Continuously updated with the current TCP A rotation (little-endian)
1064- 1065	TCP pose B (Rotation about Y-axis)	REAL, Float32 (LE)	Continuously updated with the current TCP B rotation (little-endian)
1066- 1067	TCP pose C (Rotation about Z-axis)	REAL, Float32 (LE)	Continuously updated with the current TCP C rotation (little-endian)
1435- 1450	Digital Outputs DO_0-DO_15	USINT Bit	Continuously updated with the state of DO_0 to DO_15

MODBUS HOLDING REGISTERS (READ/WRITE)

Ad- dress	Description	Type	Notes
2000	Set Digital Output DO_0	UINT16	Sets the value of digital output pin DO_0
2001	Set Digital Output DO_1	UINT16	Sets the value of digital output pin DO_1
2002	Set Digital Output DO_2	UINT16	Sets the value of digital output pin DO_2
2003	Set Digital Output DO_3	UINT16	Sets the value of digital output pin DO_3
2004	Set Digital Output DO_4	UINT16	Sets the value of digital output pin DO_4
2005	Set Digital Output DO_5	UINT16	Sets the value of digital output pin DO_5
2006	Set Digital Output DO_6	UINT16	Sets the value of digital output pin DO_6
2007	Set Digital Output DO_7	UINT16	Sets the value of digital output pin DO_7
2008	Set Digital Output DO_8	UINT16	Sets the value of digital output pin DO_8
2009	Set Digital Output DO_9	UINT16	Sets the value of digital output pin DO_9
2010	Set Digital Output DO_10	UINT16	Sets the value of digital output pin DO_10
2011	Set Digital Output DO_11	UINT16	Sets the value of digital output pin DO_11
2012	Set Digital Output DO_12	UINT16	Sets the value of digital output pin DO_12
2013	Set Digital Output DO_13	UINT16	Sets the value of digital output pin DO_13
2014	Set Digital Output DO_14	UINT16	Sets the value of digital output pin DO_14
2015	Set Digital Output DO_15	UINT16	Sets the value of digital output pin DO_15
2019	Set Analog Output AO_0	REAL	Sets the value of analog output pin AO_0
2020	Set Analog Output AO_1	REAL	Sets the value of analog output pin AO_1
2021	Set Analog Output AO_2	REAL	Sets the value of analog output pin AO_2
2022	Set Analog Output AO_3	REAL	Sets the value of analog output pin AO_3
2023	Set Analog Output AO_4	REAL	Sets the value of analog output pin AO_4
2024	Set Analog Output AO_5	REAL	Sets the value of analog output pin AO_5
2025	Set Analog Output AO_6	REAL	Sets the value of analog output pin AO_6
2026	Set Analog Output AO_7	REAL	Sets the value of analog output pin AO_7
2027	Power Off Robot	UINT16	Triggers the robot to power off
2028	Power On Robot	UINT16	Triggers the robot to power on
2029	Set Robot to Teach Mode	UINT16	Triggers the robot to switch to teach mode
2030	Set Robot to Auto Mode	UINT16	Triggers the robot to switch to automatic mode
2031	Stop GUI Program Execution	UINT16	Stops any running GUI programs
2035	Reset Error/Emergency State	UINT16	Resets the robot's error or emergency state
2037	Triggers the tree program execution mapped to the given int value	UINT16	Executes the program mapped to the given int value
2205	Proportional Gain for ServoX	FP32 (LittleEndian)	Gain for controlling the ServoX motion (range: 0.2 - 100)

continues on next page

Table 1 – continued from previous page

Ad- dress	Description	Type	Notes
2209	ServoJ/ServoX Target Position for Axis 1	FP32 (LittleEndian)	Updated with desired target position for Axis 1
2211	ServoJ/ServoX Target Position for Axis 2	FP32 (LittleEndian)	Updated with desired target position for Axis 2
2213	ServoJ/ServoX Target Position for Axis 3	FP32 (LittleEndian)	Updated with desired target position for Axis 3
2215	ServoJ/ServoX Target Position for Axis 4	FP32 (LittleEndian)	Updated with desired target position for Axis 4
2217	ServoJ/ServoX Target Position for Axis 5	FP32 (LittleEndian)	Updated with desired target position for Axis 5
2219	ServoJ/ServoX Target Position for Axis 6	FP32 (LittleEndian)	Updated with desired target position for Axis 6
2221	ServoJ/ServoX Target Position for Axis 7	FP32 (LittleEndian)	Updated with desired target position for Axis 7
2257	ServoJ/ServoX Target Velocity for Axis 1	FP32 (LittleEndian)	Updated with desired target velocity for Axis 1
2259	ServoJ/ServoX Target Velocity for Axis 2	FP32 (LittleEndian)	Updated with desired target velocity for Axis 2
2261	ServoJ/ServoX Target Velocity for Axis 3	FP32 (LittleEndian)	Updated with desired target velocity for Axis 3
2263	ServoJ/ServoX Target Velocity for Axis 4	FP32 (LittleEndian)	Updated with desired target velocity for Axis 4
2265	ServoJ/ServoX Target Velocity for Axis 5	FP32 (LittleEndian)	Updated with desired target velocity for Axis 5
2267	ServoJ/ServoX Target Velocity for Axis 6	FP32 (LittleEndian)	Updated with desired target velocity for Axis 6
2269	ServoJ/ServoX Target Velocity for Axis 7	FP32 (LittleEndian)	Updated with desired target velocity for Axis 7
2321	ServoJ/ServoX Target Acceleration for Axis 1	FP32 (LittleEndian)	Updated with desired target acceleration for Axis 1
2323	ServoJ/ServoX Target Acceleration for Axis 2	FP32 (LittleEndian)	Updated with desired target acceleration for Axis 2
2325	ServoJ/ServoX Target Acceleration for Axis 3	FP32 (LittleEndian)	Updated with desired target acceleration for Axis 3
2327	ServoJ/ServoX Target Acceleration for Axis 4	FP32 (LittleEndian)	Updated with desired target acceleration for Axis 4
2329	ServoJ/ServoX Target Acceleration for Axis 5	FP32 (LittleEndian)	Updated with desired target acceleration for Axis 5
2331	ServoJ/ServoX Target Acceleration for Axis 6	FP32 (LittleEndian)	Updated with desired target acceleration for Axis 6
2333	ServoJ/ServoX Target Acceleration for Axis 7	FP32 (LittleEndian)	Updated with desired target acceleration for Axis 7

3.1 General Purpose Registers (Custom Use)

In addition to the registers listed above, the Modbus server reserves a block of holding registers for general purpose use. These registers are available for custom application data, configuration parameters, or custom commands.

- **Address Range:** 2500 to 2599 (100 registers)
- **Data Type:** By default, each register is a UINT16 (16-bit integer)
- **Notes:** Clients can read from or write to these registers freely without interfering with the predefined robot control registers.